



与命名的强制类型转换相比，旧式的强制类型转换从表现形式上来说不那么清晰明了，容易被看漏，所以一旦转换过程出现问题，追踪起来也更加困难。

4.11.3 节练习

练习 4.36: 假设 i 是 int 类型，d 是 double 类型，书写表达式 i*=d 使其执行整数类型的乘法而非浮点类型的乘法。

练习 4.37: 用命名的强制类型转换改写下列旧式的转换语句。

```
int i; double d; const string *ps; char *pc; void *pv;
(a) pv = (void*)ps; (b) i = int(*pc);
(c) pv = &d; (d) pc = (char*) pv;
```

练习 4.38: 说明下面这条表达式的含义。

```
double slope = static_cast<double>(j/i);
```

4.12 运算符优先级表

表 4.4: 运算符优先级

结合律和运算符		功能	用法	参考页码
左	::	全局作用域	::name	256
左	::	类作用域	class::name	79
左	::	命名空间作用域	namespace::name	74
左	.	成员选择	object.member	20
左	->	成员选择	pointer->member	98
左	[]	下标	expr[expr]	104
左	()	函数调用	name(expr_list)	20
左	()	类型构造	type(expr_list)	145
右	++	后置递增运算	lvalue++	131
右	--	后置递减运算	lvalue--	131
右	typeid	类型 ID	typeid(type)	731
右	typeid	运行时类型 ID	typeid(expr)	731
右	explicit cast	类型转换	cast_name<type>(expr)	144
右	++	前置递增运算	++lvalue	131
右	--	前置递减运算	--lvalue	131
右	~	位求反	~expr	136
右	!	逻辑非	!expr	126
右	-	一元负号	-expr	124
右	+	一元正号	+expr	124
右	*	解引用	*expr	48
右	&	取地址	&lvalue	47
右	()	类型转换	(type) expr	145
右	sizeof	对象的大小	sizeof expr	139

续表

结合律和运算符	功能	用法	参考页码
右 sizeof	类型的大小	sizeof(type)	139
右 sizeof...	参数包的大小	sizeof...(name)	619
右 new	创建对象	new type	407
右 new[]	创建数组	new type[size]	407
右 delete	释放对象	delete expr	409
右 delete[]	释放数组	delete[] expr	409
右 noexcept	能否抛出异常	noexcept (expr)	690
左 ->*	指向成员选择的指针	ptr->*ptr_to_member	740
左 .*	指向成员选择的指针	obj.*ptr_to_member	740
左 *	乘法	expr * expr	124
左 /	除法	expr / expr	124
左 %	取模（取余）	expr % expr	124
左 +	加法	expr + expr	124
左 -	减法	expr - expr	124
左 <<	向左移位	expr << expr	136
左 >>	向右移位	expr >> expr	136
左 <	小于	expr < expr	126
左 <=	小于等于	expr <= expr	126
左 >	大于	expr > expr	126
左 >=	大于等于	expr >= expr	126
左 ==	相等	expr == expr	126
左 !=	不相等	expr != expr	126
左 &	位与	expr & expr	136
左 ^	位异或	expr ^ expr	136
左	位或	expr expr	136
左 &&	逻辑与	expr && expr	126
左	逻辑或	expr expr	126
右 ? :	条件	expr ? expr : expr	134
右 =	赋值	lvalue = expr	129
右 *=, /=, %=	复合赋值	lvalue += expr 等	129
右 +=, -=			129
右 <<=, >>=			129
右 &=, =, ^=			129
右 throw	抛出异常	throw expr	173
左 ,	逗号	expr, expr	140

小结

168

C++语言提供了一套丰富的运算符，并定义了这些运算符作用于内置类型的运算对象时所执行的操作。此外，C++语言还支持运算符重载的机制，允许我们自己定义运算符作用于类类型时的含义。第14章将介绍如何定义作用于用户类型的运算符。

对于含有超过一个运算符的表达式，要想理解其含义关键要理解优先级、结合律和求值顺序。每个运算符都有其对应的优先级和结合律，优先级规定了复合表达式中运算符组合的方式，结合律则说明当运算符的优先级一样时应该如何组合。

大多数运算符并不明确规定运算对象的求值顺序：编译器有权自由选择先对左侧运算对象求值还是先对右侧运算对象求值。一般来说，运算对象的求值顺序对表达式的最终结果没有影响。但是，如果两个运算对象指向同一个对象而且其中一个改变了对象的值，就会导致程序出现不易发现的严重缺陷。

最后一点，运算对象经常从原始类型自动转换成某种关联的类型。例如，表达式中的小整型会自动提升成大整型。不论内置类型还是类类型都涉及类型转换的问题。如果需要，我们还可以显式地进行强制类型转换。

术语表

算术转换 (arithmetic conversion) 从一种算术类型转换成另一种算术类型。在二元运算符的上下文中，为了保留精度，算术转换通常把较小的类型转换成较大的类型（例如整型转换成浮点型）。

结合律 (associativity) 规定具有相同优先级的运算符如何组合在一起。结合律分为左结合律（运算符从左向右组合）和右结合律（运算符从右向左组合）。

二元运算符 (binary operator) 有两个运算对象参与运算的运算符。

强制类型转换 (cast) 一种显式的类型转换。

复合表达式 (compound expression) 含有多于一个运算符的表达式。

const_cast 一种涉及 const 的强制类型转换。将底层 const 对象转换成对应的非常量类型，或者执行相反的转换。

转换 (conversion) 一种类型的值改变成另一种类型的值的过程。C++语言定义了内置类型的转换规则。类类型同样可以转换。

dynamic_cast 和继承及运行时类型识别一

起使用。参见 19.2 节（第 730 页）。

表达式 (expression) C++程序中最低级别的计算。表达式将运算符作用于一个或多个运算对象，每个表达式都有对应的求值结果。表达式本身也可以作为运算对象，这时就得到了对多个运算符求值的复合表达式。

隐式转换 (implicit conversion) 由编译器自动执行的类型转换。假如表达式需要某种特定的类型而运算对象是另外一种类型，此时只要规则允许，编译器就会自动地将运算对象转换成所需的类型。

整型提升 (integral promotion) 把一种较小的整数类型转换成与之最接近的较大整数类型的过程。不论是否真的需要，小整数类型（即 short、char 等）总是会得到提升。

左值 (lvalue) 是指那些求值结果为对象或函数的表达式。一个表示对象的非常量左值可以作为赋值运算符的左侧运算对象。

运算对象 (operand) 表达式在某些值上执行运算，这些值就是运算对象。一个运算

169